

DEPARTMENT OF COMPUTER SCIENCE



University of
Reading

CS1IP: Imperative Programming

Week 4: Functions

Learning Outcomes

- What are functions?
- What is about recursion?
- What is the importance of composition reasoning and tests?

What is a program?

A sequence of **instructions** that is **executed** by a **computer**.

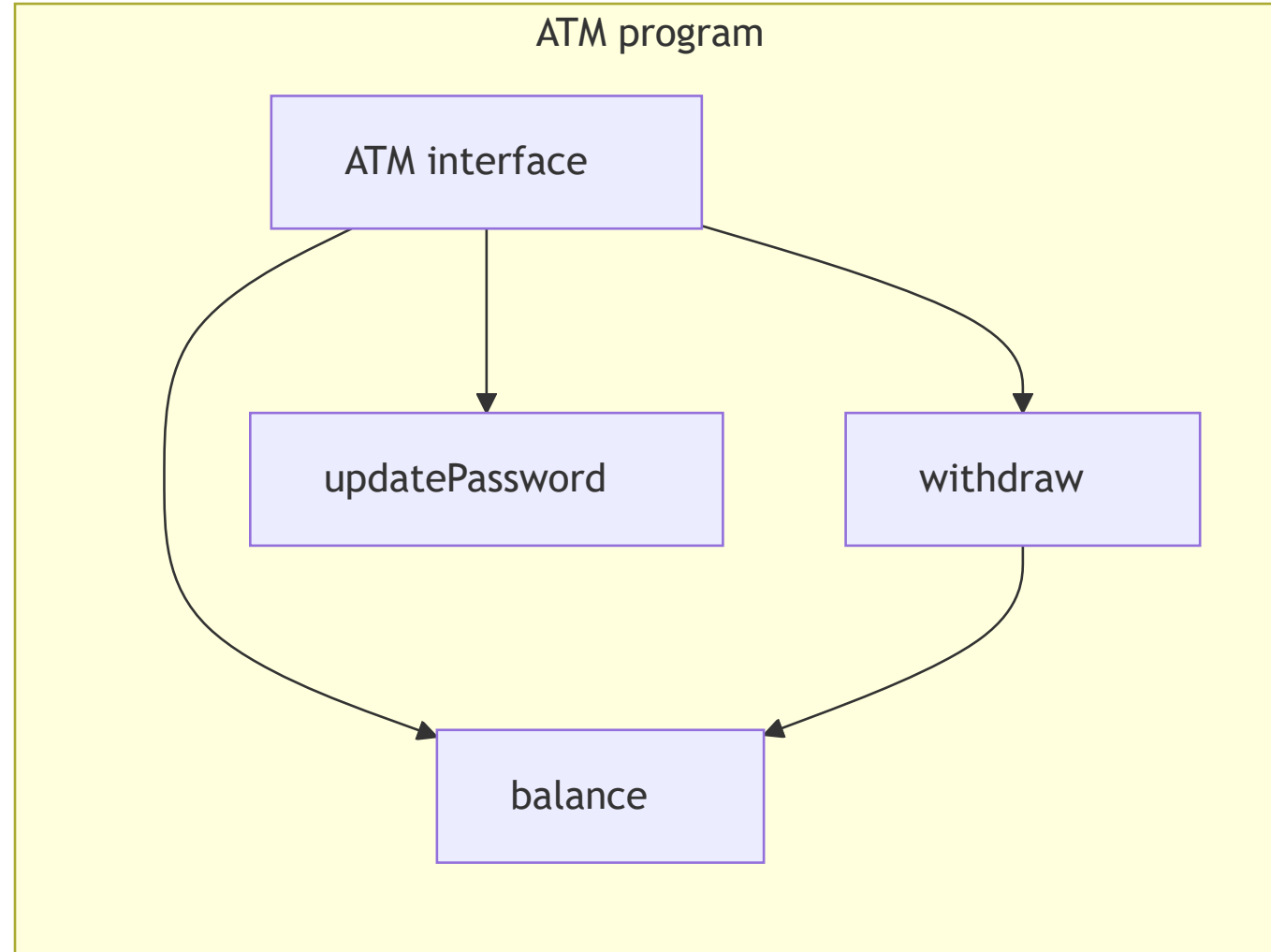
These instructions can be organized in functions

What is a function?

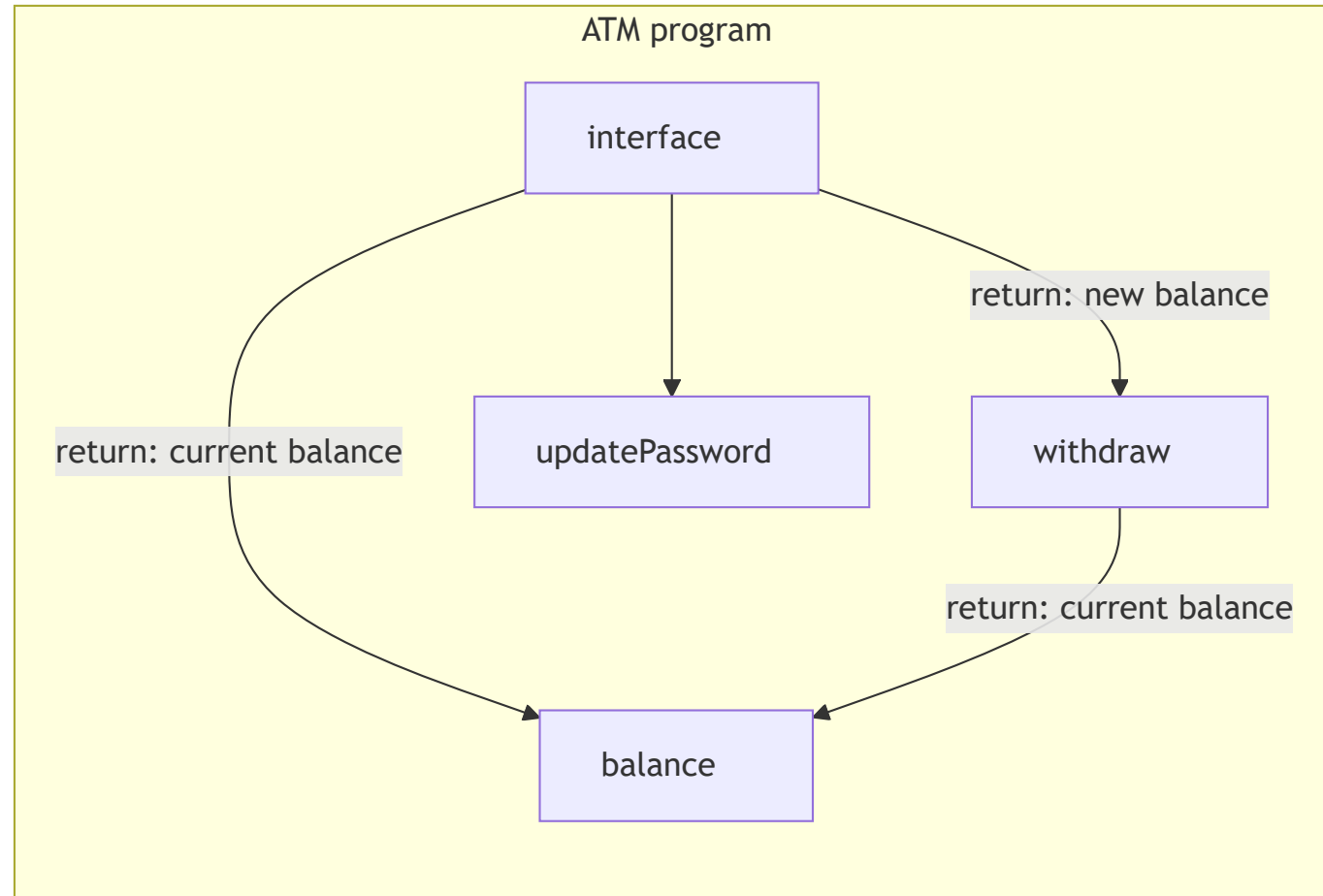
A block of code designed to perform a specific task.

- Functions increase modularity and reusability.
- Function can also be seen as `subprograms` that are part of a program.
- A function received input as `params` and produce a `return`
 - Functions can communicate!

Let imagine an ATM program



Let imagine a ATM program: functions returns



Syntax of a Function in Java

```
static RETURN_TYPE functionName(parameters) {  
    // Code block  
    return value;  
}
```

Example

```
static int add(int a, int b) {  
    return a + b;  
}
```

Calling a function

- A function is called using its name and passing the required arguments.

Example

```
static int add(int a, int b) {  
    return a + b;  
}  
...  
add(5, 3); // return 8
```

Trace

1. Function `add` is called with arguments `5` and `3`.
2. Function executes and returns `8`.

Calling a function from Java `main()`

```
public class W4Lecture {  
    static int add(int a, int b) {  
        return a + b;  
    }  
    public static void main(String args[]) {  
        System.out.println(add(2, 2));  
    }  
}
```

Note: You are required to `static` when using java in imperative way. When using Java in as object-oriented programming you can use without `static`. You will see more in CS10P.

Calling Functions without return

- A function can also has no return with the keyword `void`

Example

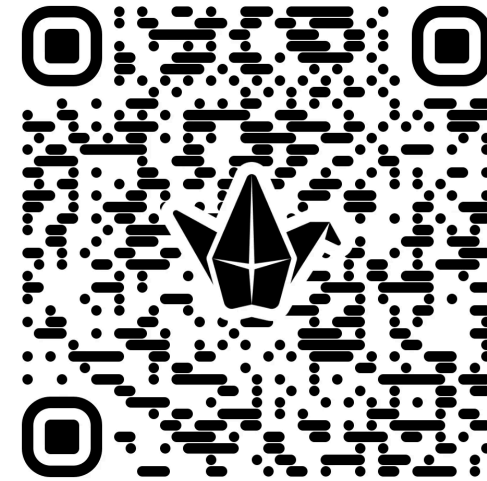
```
static void print(int x) {  
    System.out.println(x); // This will print 5  
}  
public static void main(String args[]) {  
    print(5)  
}
```

Quiz 1 ([click here](#) or scan QRCode)

Which is the output of the following code?

```
public class W4Lecture {  
    static int add(int a, int b) {  
        return a + b;  
    }  
    public static void main(String args[]) {  
        System.out.println(add(2.5, 2));  
    }  
}
```

- a. 4
- b. 4.5
- c. 5
- d. Compilation error



Function Overloading

It a way to define multiple functions with the same name but different parameter lists.

```
public class W4Lecture {  
    static int add(int a, int b) {  
        return a + b;  
    }  
    static double add(double a, double b) {  
        return a + b;  
    }  
    public static void main(String args[]) {  
        System.out.println( add(2.5, 2) );  
    }  
}
```

Note: The compiler selects the correct version based on argument types

Function Overloading

Which is the output of the following code?

```
public class W4Lecture {  
    int add(int a, int b) {  
        return a + b;  
    }  
    double add(double a, double b) {  
        return a + b;  
    }  
    public static void main(String args[]) {  
        System.out.println( add(2, 2) );  
    }  
}
```

Variable Scope in a Function

- **Variable Scope** refers to the visibility
- In Java, the scope is primarily determined by where the variable is declared.

Global Scope:

- For instance, **variables declared in the start of the document**
- Accessible among all functions where the variable was created
- NOTE: When using Java classes, such variables use the `static` keyword.

Local Scope:

- **Variables declared inside functions or code blocks.**
- Accessible only within that method or block.

Global Scope variable

```
public class W4Lecture {  
    static int add(int a, int b) {  
        return a + b;  
    }  
    public static void main(String args[]) {  
        System.out.println(add(2, 2));  
    }  
}
```

Local Scope variable inside a function

```
public class W4Lecture {  
    static int x = 5;  
    static void print() {  
        static int y = 6;  
        System.out.println(y); // This will print 6  
    }  
    public static void main(String args[]) {  
        print();  
    }  
}
```

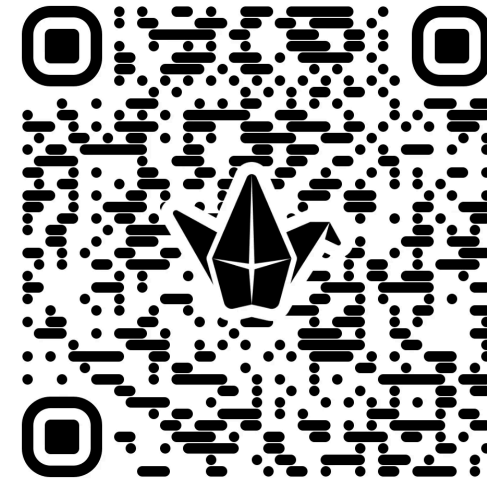
Local Scope variable of a function param

```
public class W4Lecture {  
    static int x = 5;  
    static void print(int x) {  
        System.out.println(x); // This will print 6  
    }  
    public static void main(String args[]) {  
        print(6);  
    }  
}
```

Quiz 2 ([click here](#) or scan QRCode)

Which is the output of the code?

```
public class W4Lecture {  
    static int x = 5;  
    static void print(int z) {  
        int y = x;  
        System.out.println(y);  
    }  
    public static void main(String args[]) {  
        print(6);  
    }  
}
```



- 6
- 5
- 0

What is recursion?



What is recursion?

Recursion occurs when a function **calls itself** to solve a problem.

It typically involves:

1. **Base Case:** The simplest case that stops recursion.
 2. **Recursive Case:** The function calls itself with a simpler problem.
- It needs to be used with care, for instance memory problem due to excessive recursive calls in the same function

Recursion example: factorial

Mathematical Definition

The **factorial** of a non-negative integer (n), denoted as ($n!$), is defined as:

$$\begin{aligned} n! &= \\ 1 &\text{ if } n = 0, \\ n \times (n - 1)! &\text{ if } n > 0. \end{aligned}$$

Example

$$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$$

Recursion example: factorial

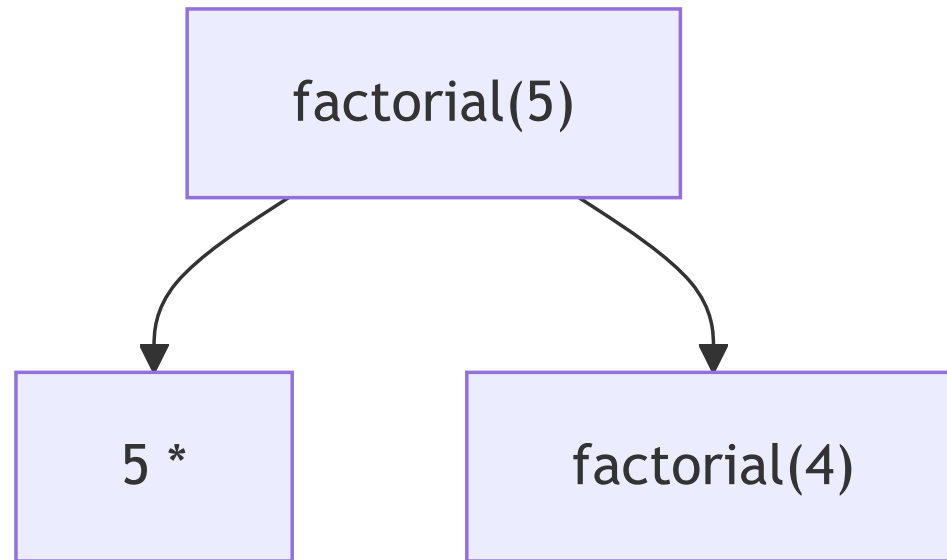
Explanation

- Base Case: return 1 when `n == 0`
- Recursive Case: return `n * factorial(n - 1)`

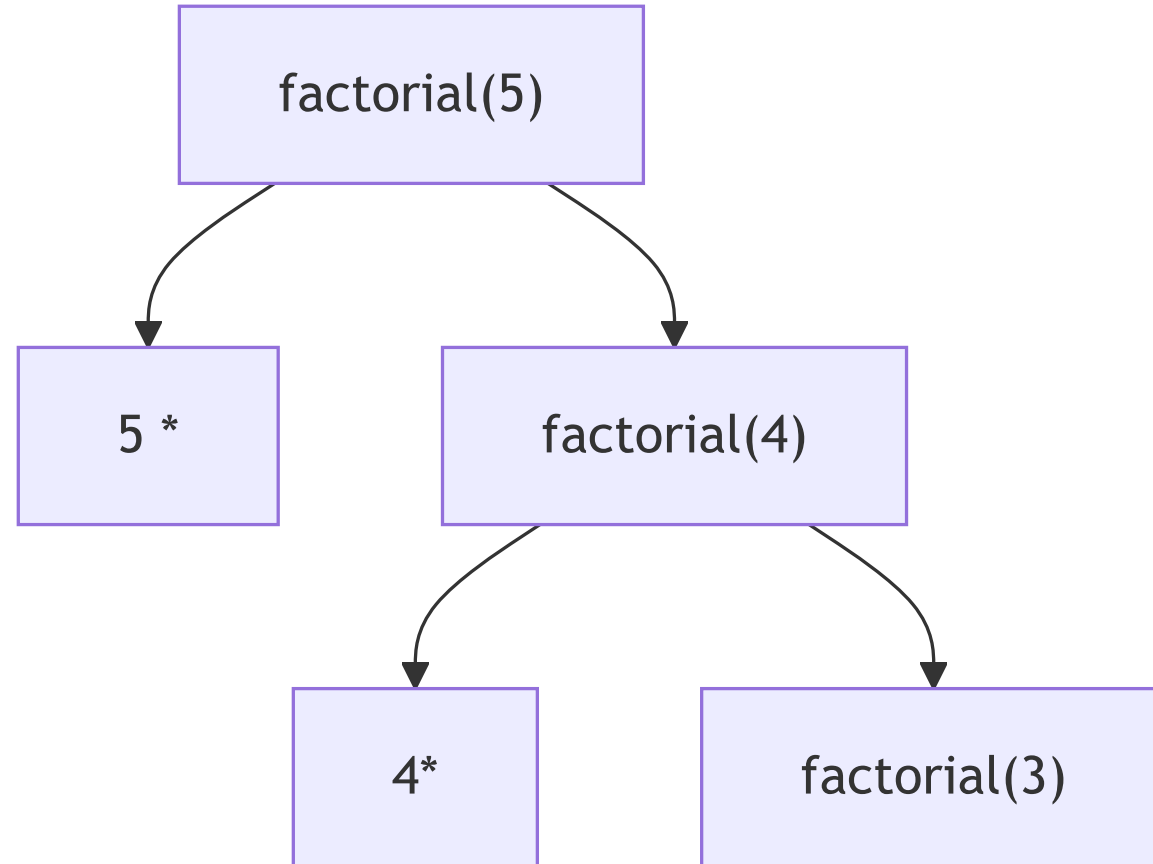
java code

```
static int factorial(int n) {  
    if (n == 0) return 1; // Base case  
    return n * factorial(n - 1); // Recursive case  
}
```

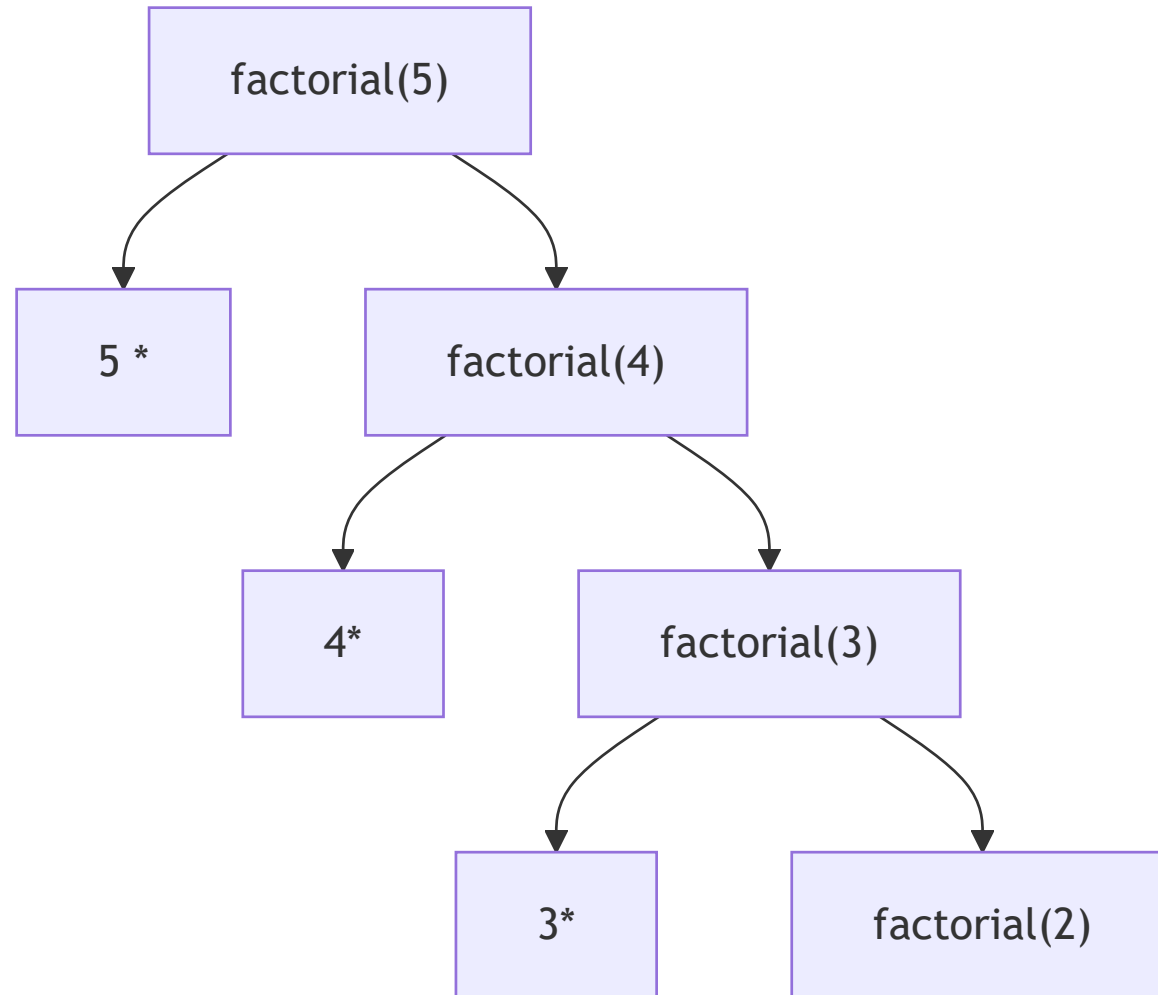
Recursion example: factorial



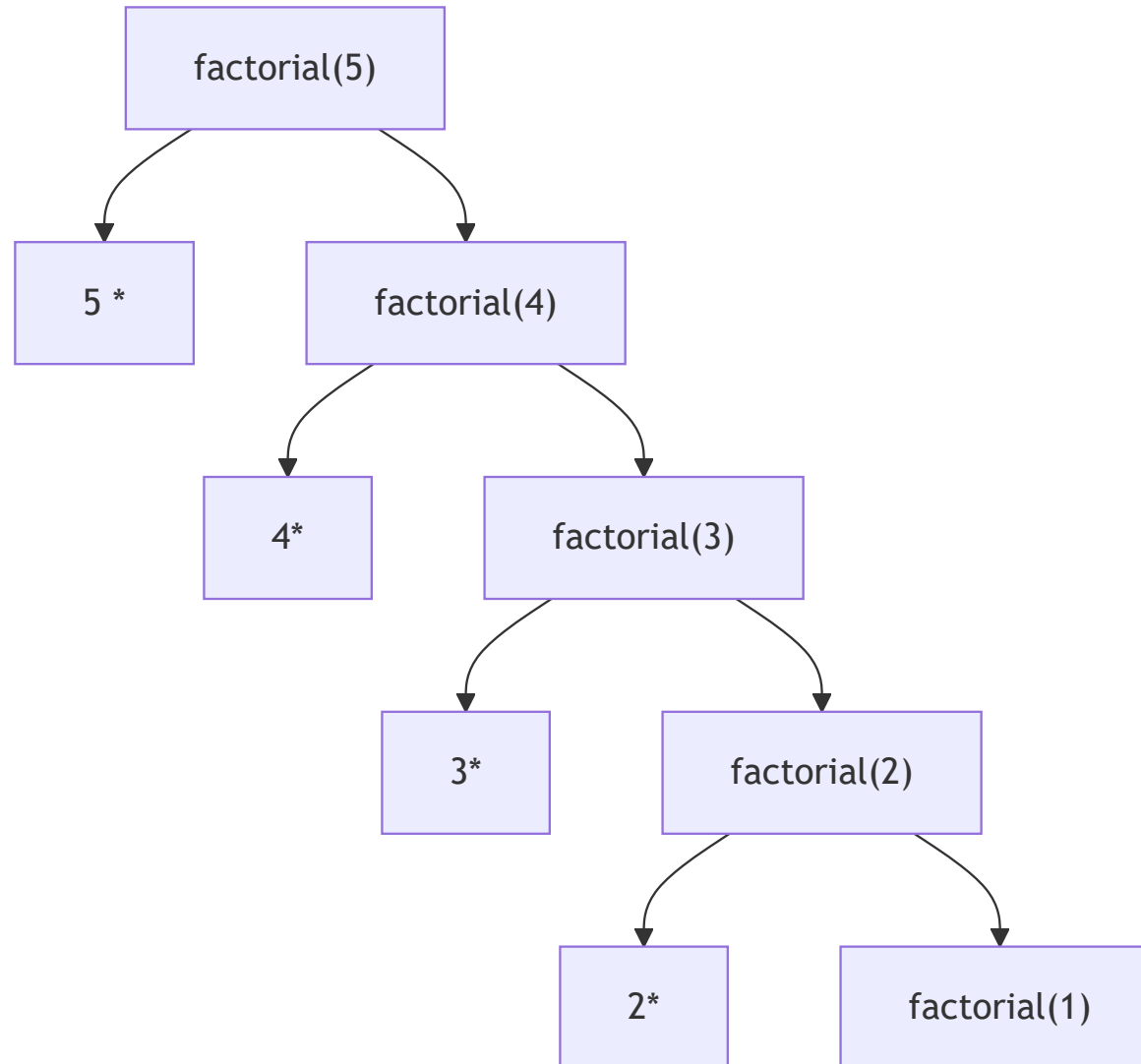
Recursion example: factorial



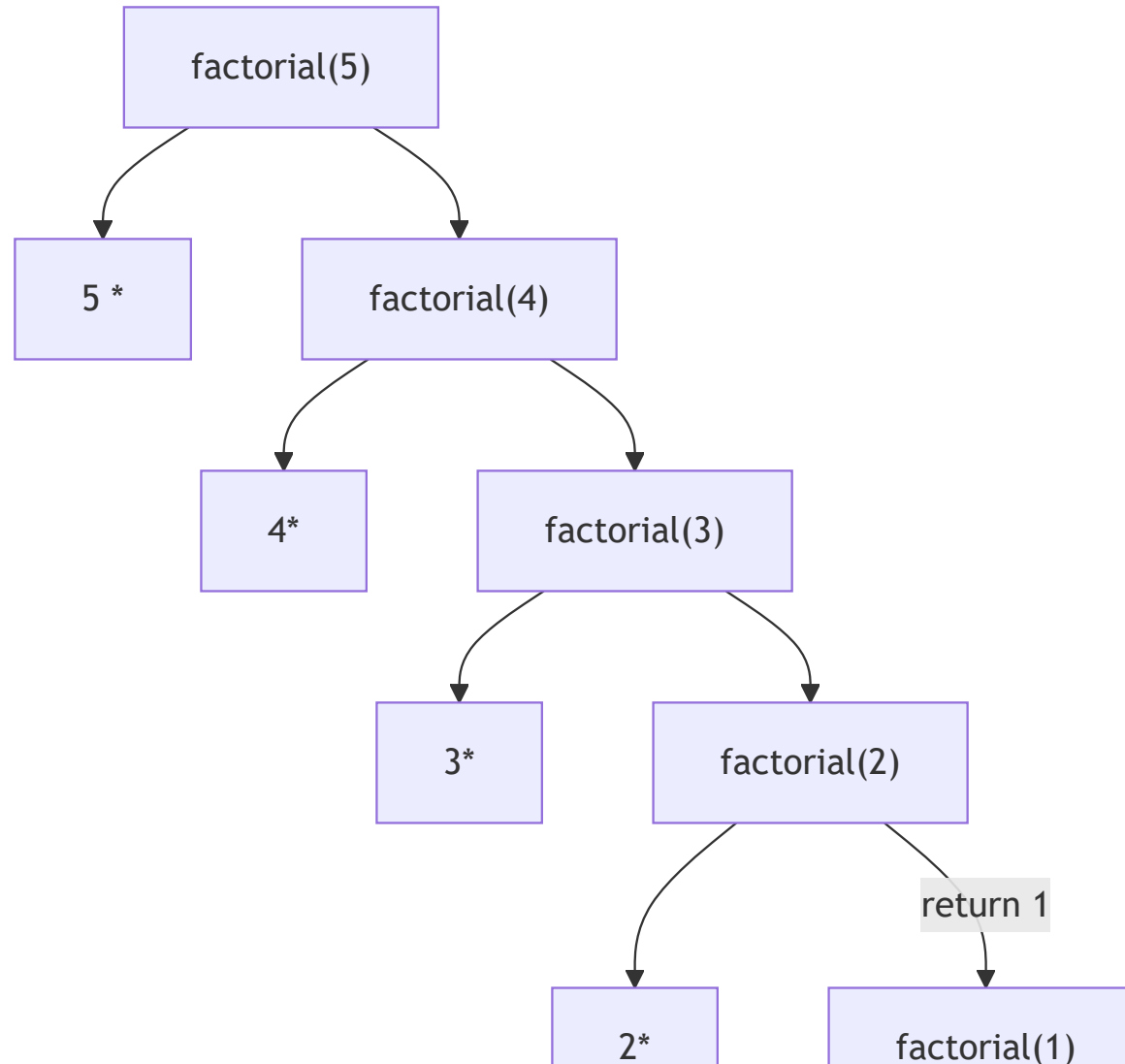
Recursion example: factorial



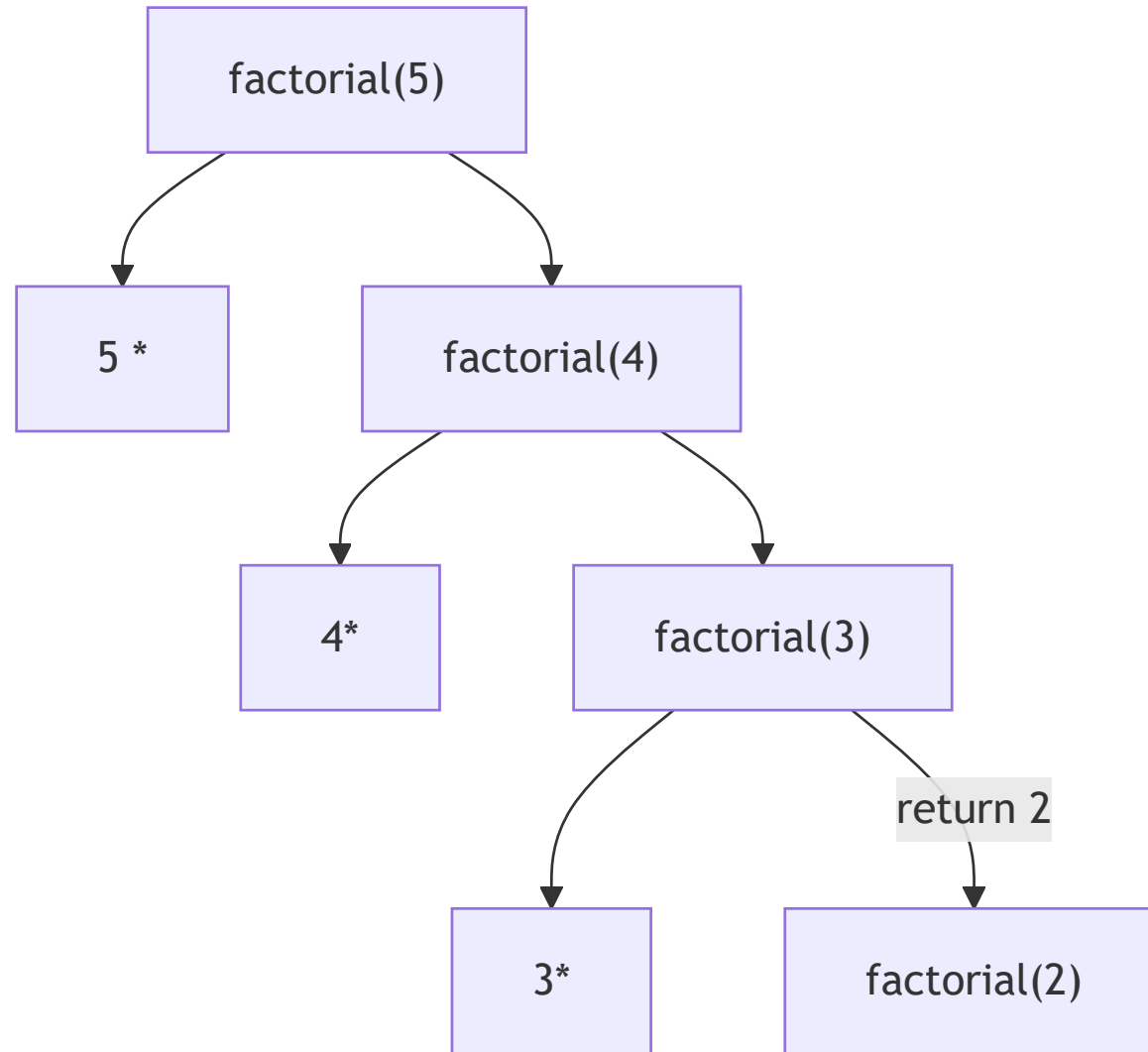
Recursion example: factorial



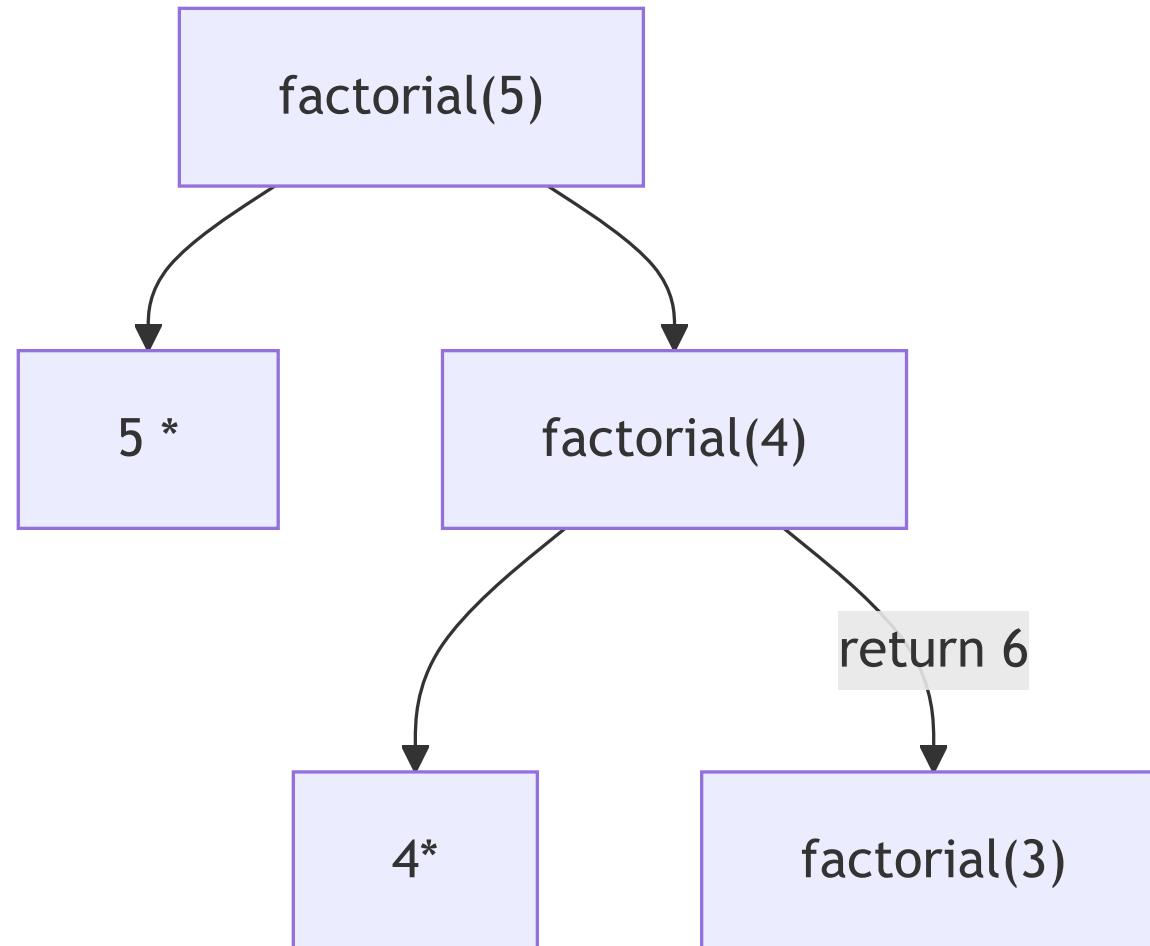
Recursion example: factorial



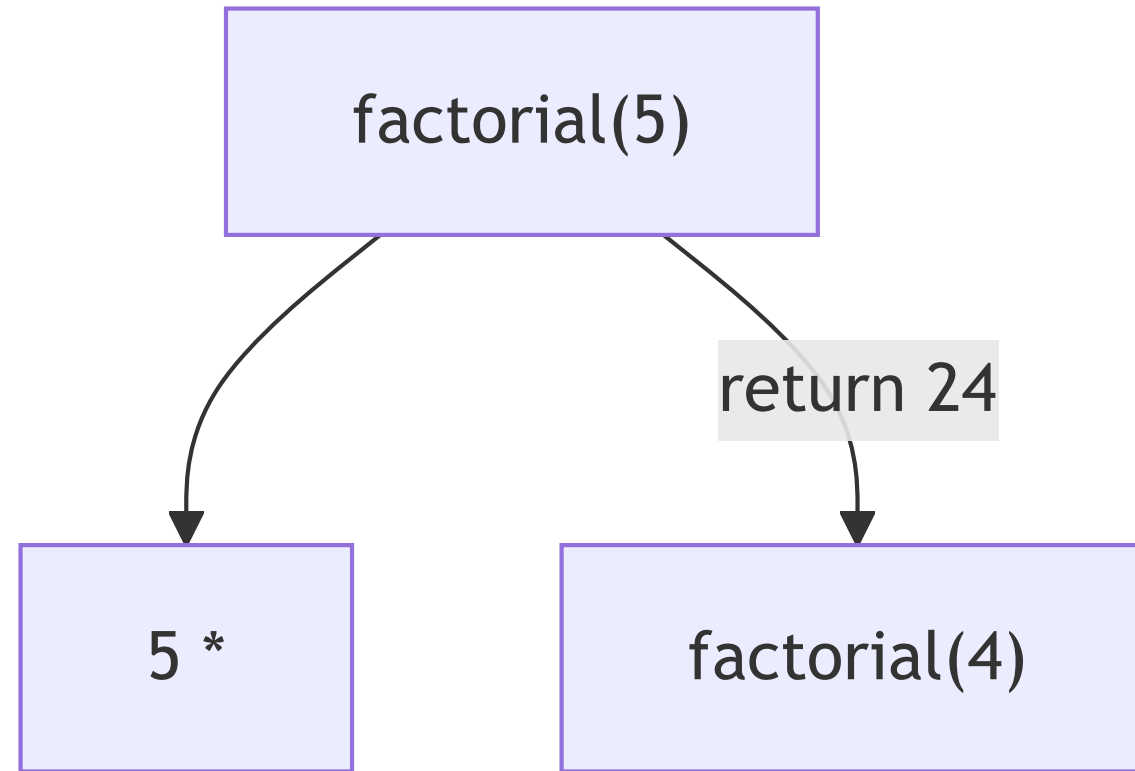
Recursion example: factorial



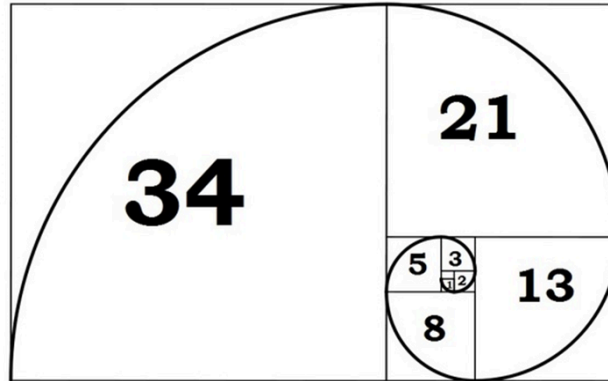
Recursion example: factorial



Recursion example: factorial



Recursion example: Fibonacci



Mathematical Definition

$$fib(n) = 0 \text{ if } n = 0$$

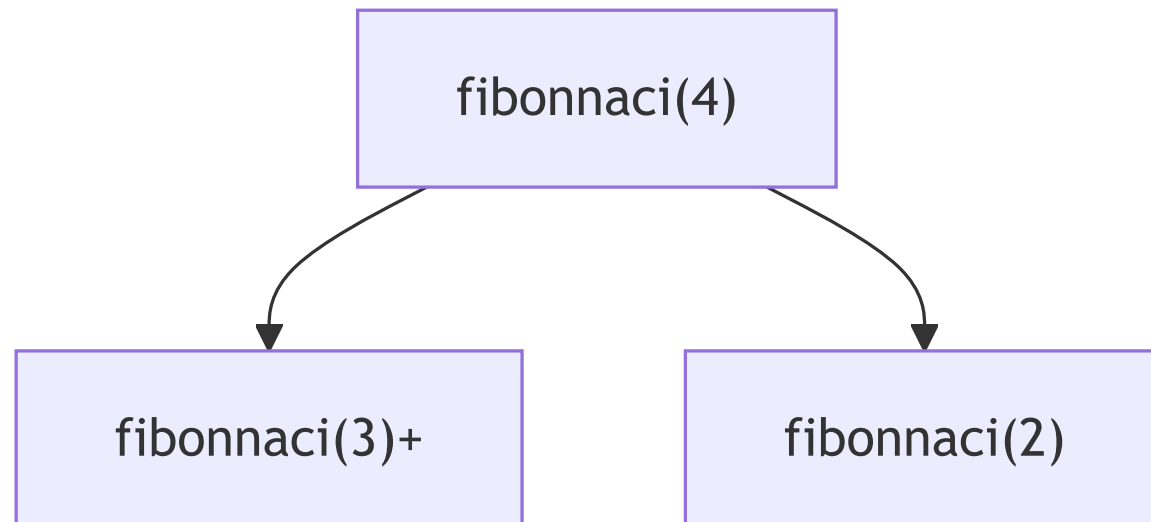
$$fib(n) = 1 \text{ if } n = 1$$

Recursion example: Fibonacci

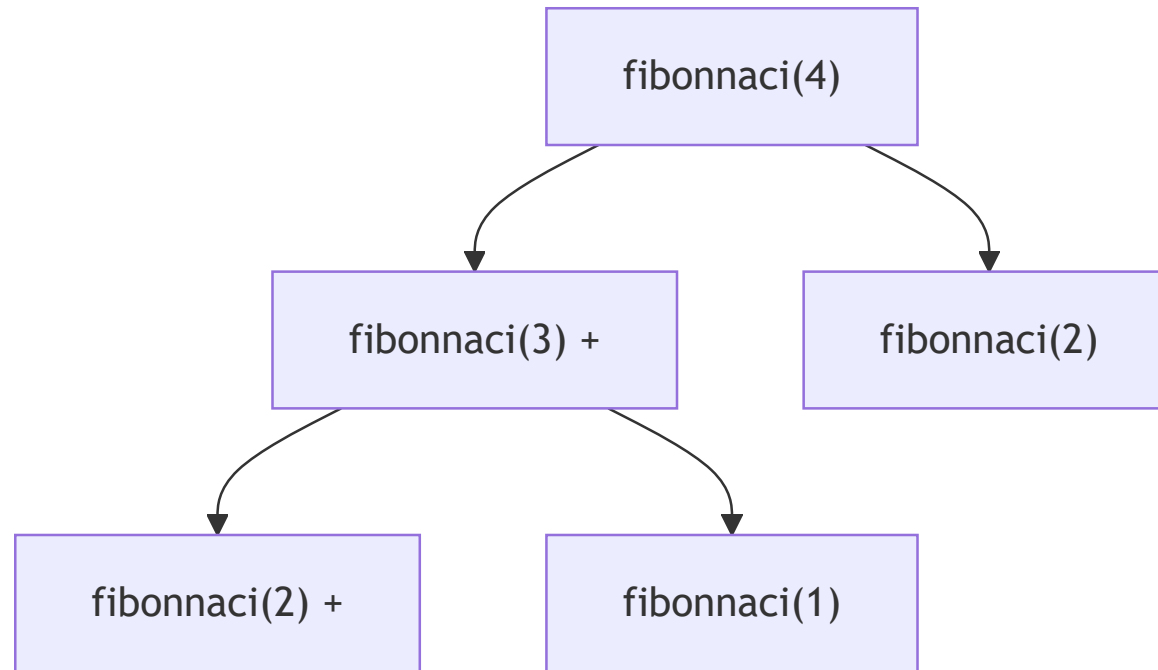
Explanation

- Base Case: return `0` when `n == 0` or `1` if `n == 1`
- Recursive Case: return `fib(n - 1) + fib(n - 2)`

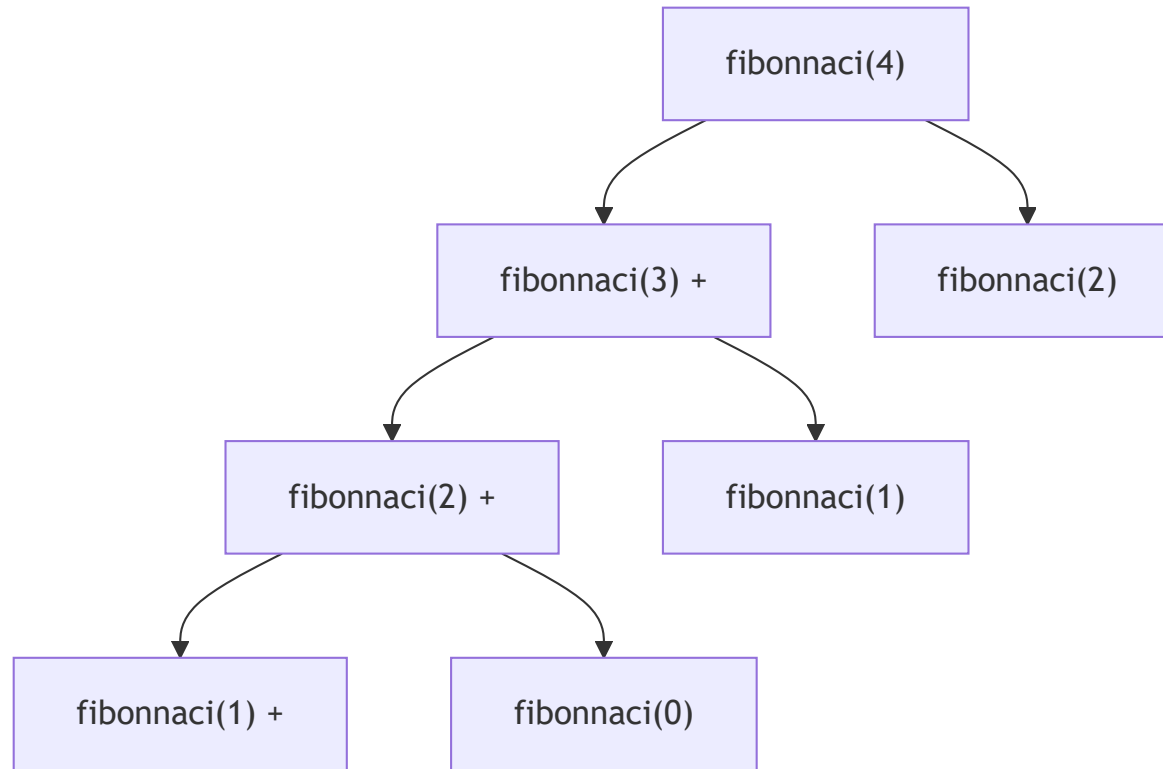
java code



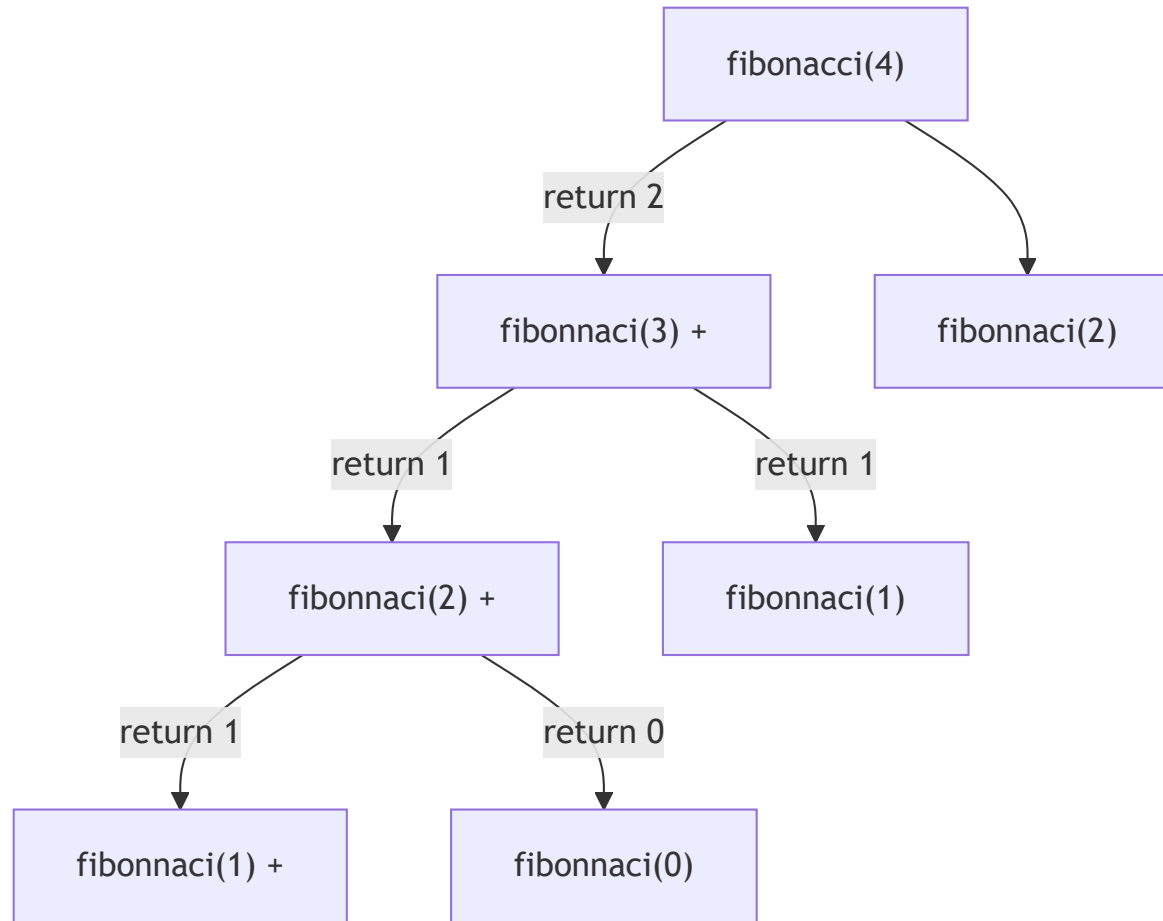
Recursion example: Fibonacci



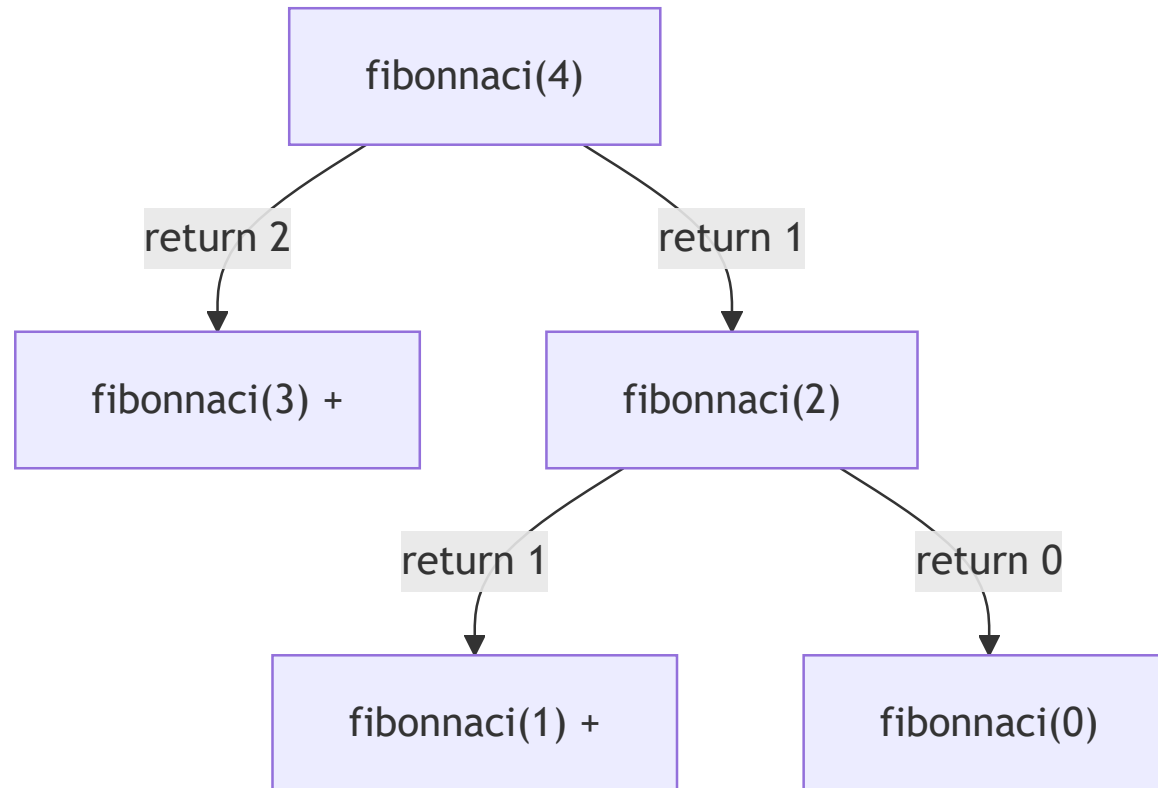
Recursion example: Fibonacci



Recursion example: Fibonacci



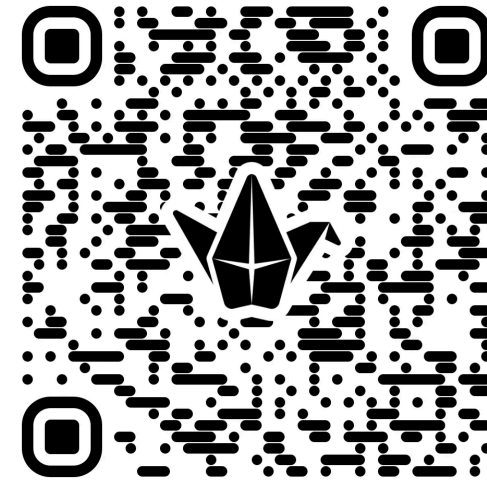
Recursion example: Fibonacci



Quiz 3 ([click here](#) or scan QRCode)

Identify the error in the recursive function

```
public class W4Lecture {  
    static int sum(int n) {  
        return n + sum(n - 1);  
    }  
    public static void main(String args[]) {  
        sum(6);  
    }  
}
```



- n should be incremented, not decremented
- It will return 0 for all values of n
- The base case is missing
- It only works for positive numbers

Function testing using `assert`

What is `assert`?

- `assert` is commonly used for **quick tests** during development or debugging.

The `assert` statement is used to test whether a condition in your code

- If the condition is `false`, it stops with a error **AssertionError**.
- If the condition is `true`, the code continues

```
int x = 5;  
assert x > 0;  
System.out.println(x) // only reach if above assert is valid
```

Note: Other types of tests includes **Unit Testing** using `JUnit`, will not be covered at CS1IP.

assert for function testing

The `assert` is useful for **function testing** to check whether the function behaves as expected.

```
public class W4Lecture {  
    static int add(int a, int b) {  
        return a + b;  
    }  
    public static void main(String args[]) {  
        assert add(0, 0) == 0;  
        assert add(1, 1) == 2;  
        assert add(2, 3) == 5;  
    }  
}
```

assert for function testing

```
public class W4Lecture {  
    static int square(int num) {  
        return num * num;  
    }  
    public static void main(String args[]) {  
        assert square(2) == 4;    // Valid case  
        assert square(-1) == 1;  // What happens? Should we restrict negative input?  
    }  
}
```

How enable `assert` in VSCode

Setup Environment to use `assert`

To enable `assert` the Java runtime need to have the `-ea` param.

On VSCode you can set the param by: `Ctrl+Shift+P` to open actions -> type/choose

Preferences: Open User Settings (JSON) -> then add the line

```
"java.debug.settings.vmArgs": "-ea",
```

VSCode demo

Testing functions specifications

A **specification** is a detailed description of what a function should do, including inputs, outputs, and behaviour.

- **Preconditions:** What must be true before the function is called.
- **Postconditions:** What will be true after the function completes.

It should be written as **comments or documentation**.

Testing functions specifications

```
/**
 * Adds two integers and returns the result.
 * param a - the first integer between 0 and 1
 * param b - the second integer between 0 and 1
 * return the sum of a and b
 */
static int add(int a, int b) {
    ...
}
```

Testing functions specifications

```
/**
 * Adds two integers and returns the result.
 * param a - the first integer between 0 and 1
 * param b - the second integer between 0 and 1
 * return the sum of a and b
 */
static int add(int a, int b) {
    ...
}
public static void main(String args[]) {
    assert add(0,0) == 0;
    assert add(0,1) == 1;
    assert add(1,0) == 1;
    assert add(0,1) == 1;
}
```

assert to check pre conditions

```
static double divide(double a, double b) {  
    return a * b;  
}  
public static void main(String args[]) {  
    divide(10, 0);  
}
```

assert to check pre conditions

```
static double divide(double a, double b) {  
    assert b != 0  
    return a * b;  
}  
public static void main(String args[]) {  
    divide(10, 0);  
}
```

Importance of Testing

- Ensures code correctness.
- Identifies edge cases and errors early.
- Saves time in debugging and maintenance.

Review Learning Outcomes

Review Learning Outcomes

What are functions?

- **Functions** Allow modular design and reuse.

What is about recursion?

- **Recursion** is when a function call itself

What is the importance testing?

- **Testing** Ensure code correctness and make code easier to maintain.

DEPARTMENT OF COMPUTER SCIENCE



Thank you! 😊

Do you have any questions 🙋 ?

The next lecture will cover: Arrays